

AMPLIACIÓN DEL MODELO DE ATRIBUTOS FLEXIBLES DE OMNIMERGE

1. Introducción

El módulo de entidades de OmniMerge debe permitir que cada usuario construya libremente sus propias entidades, atributos y valores.

Para conseguir esta flexibilidad, no basta con almacenar:

Entidad → Atributo → Valor

También es necesario representar:

- Valores seleccionables mediante catálogos.
- Atributos libres.
- Atributos con uno o varios valores.
- Grupos de atributos.
- Orden de presentación.
- Jerarquías entre atributos.
- Dependencias entre atributos.
- Condiciones basadas en valores seleccionados.
- Atributos que solo puedan utilizarse dentro de determinados contextos.
- Relaciones entre atributos de distintos tipos.

Por ejemplo:

Entidad: Naruto Uzumaki

Obra o anime:

- Naruto

Tipo de chakra:

- Viento

Aldea:

- Aldea Oculta de la Hoja

Rango:

- Genin

Habilidades:

- Rasengan

- Clones de sombra

En este caso, el atributo **Tipo de chakra** podría mostrarse solamente cuando la entidad pertenece al anime **Naruto**.

Esto no debe representarse únicamente mediante una columna de jerarquía. Se necesita una estructura más completa de relaciones y reglas.

2. Tabla para valores seleccionables

2.1. Idea planteada

Se propone crear una tabla que almacene valores seleccionables para determinados atributos.

Ejemplo:

Atributo: Elemento

Valores disponibles:

- Fuego
- Agua
- Tierra
- Aire

Otro ejemplo:

Atributo: Rareza

Valores disponibles:

- Común
- Raro
- Épico
- Legendario

El usuario no tendría que escribir manualmente el valor cada vez. Solo seleccionaría uno o varios valores del catálogo.

2.2. ¿Debe llamarse **adjetivo**?

La palabra **adjetivo** puede funcionar en algunos casos:

Personalidad:

- Valiente
- Amable
- Impulsivo

Sin embargo, no todos los valores seleccionables son adjetivos.

Ejemplos:

Elemento:

- Fuego
- Agua

Anime:

- Naruto
- Dragon Ball

País:

- Perú
- Japón

Tipo de chakra:

- Fuego
- Agua
- Tierra

Por eso, el nombre más general recomendado es:
`attribute_options`

En español:
`opciones_atributo`

También podría llamarse:
`catalogo_valores_atributo`

La denominación más clara para Laravel sería:
`attribute_options`

Modelo:
`AttributeOption`

En la interfaz puede mostrarse como:
Opciones
Valores disponibles
Catálogo del atributo

3. Tabla **`attribute_options`**

3.1. Propósito

Almacena los valores seleccionables pertenecientes a un atributo.
Cada atributo puede tener su propio catálogo.

Ejemplo:
Atributo: Elemento

Catálogo:
- Fuego
- Agua
- Tierra
- Aire
- Luz
- Oscuridad

Una entidad puede seleccionar solamente algunos de ellos:
Entidad: Pyron

Elemento:

- Fuego
- Luz

3.2. Campos recomendados

Campo	Tipo	Descripción
id	BIGINT, PK	Identificador de la opción
attribute_id	BIGINT, FK	Atributo al que pertenece
code	VARCHAR(50)	Código interno
name	VARCHAR(150)	Nombre de la opción
description	TEXT	Descripción
image	VARCHAR(255)	Imagen opcional
icon	VARCHAR(100)	Icono
color	VARCHAR(20)	Color representativo
numeric_value	DECIMAL(15,4)	Valor numérico opcional
parent_option_id	BIGINT, FK, NULL	Opción superior
sort_order	INT	Orden
status	VARCHAR(20)	Estado
metadata	JSON	Información adicional
created_at	TIMESTAMP	Creación
updated_at	TIMESTAMP	Actualización
deleted_at	TIMESTAMP	Borrado lógico

3.3. Ejemplo

attribute_id: Elemento

code: FIRE

name: Fuego

description: Afinidad con el fuego y el calor.

color: #FF4500

sort_order: 1

status: ACTIVE

3.4. Opciones jerárquicas

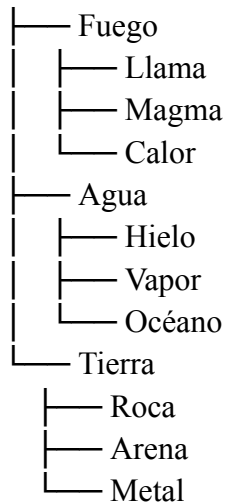
La columna:

`parent_option_id`

permite crear jerarquía dentro del catálogo de valores.

Ejemplo:

Elemento



En este caso:

Fuego

`parent_option_id = NULL`

Magma

`parent_option_id = Fuego`

Esto es una jerarquía entre opciones, no entre atributos.

4. Tipos de atributos respecto a sus valores

No todos los atributos deben utilizar un catálogo.

La tabla `attributes` necesita indicar cómo obtiene sus valores.

Se recomienda agregar:

`value_source`

Valores posibles:

FREE

CATALOG

MIXED

REFERENCE

CALCULATED

4.1. FREE

El usuario escribe el valor.

Ejemplo:

Biografía:

Naruto es un ninja perteneciente a la Aldea de la Hoja.

4.2. CATALOG

El usuario solamente selecciona opciones existentes.

Ejemplo:

Elemento:

☒ Fuego

☐ Agua

☐ Tierra

4.3. MIXED

El usuario puede seleccionar opciones o escribir un valor nuevo.

Ejemplo:

Profesión:

- Guerrero

- Mago

- Científico

- Otro: Investigador dimensional

4.4. REFERENCE

El valor es otra entidad.

Ejemplo:

Maestro:

- Jiraiya

Jiraiya no sería un texto simple, sino una referencia hacia otra entidad registrada.

4.5. CALCULATED

El valor se genera mediante una fórmula.

Ejemplo:

Poder total =

fuerza + velocidad + resistencia

Este tipo puede incorporarse en una versión futura.

5. Campos ampliados de attributes

La tabla debe incluir:

Campo	Tipo	Descripción
id	BIGINT, PK	Identificador
user_id	BIGINT, FK	Propietario
code	VARCHAR(50)	Código
name	VARCHAR(150)	Nombre
slug	VARCHAR(180)	Identificador amigable
description	TEXT	Descripción
data_type	VARCHAR(30)	Tipo de dato
value_source	VARCHAR(30)	Origen del valor
allows_multiple	BOOLEAN	Permite varios valores
allows_custom_values	BOOLEAN	Permite valores nuevos
is_required	BOOLEAN	Es obligatorio
is_filterable	BOOLEAN	Puede usarse en filtros
is_comparable	BOOLEAN	Puede compararse
is_searchable	BOOLEAN	Participa en búsquedas
is_visible	BOOLEAN	Visible por defecto
is_featured	BOOLEAN	Atributo destacado
min_numeric_value	DECIMAL	Mínimo
max_numeric_value	DECIMAL	Máximo
unit	VARCHAR(30)	Unidad
sort_order	INT	Orden general
hierarchy_level	INT	Nivel orientativo
status	VARCHAR(20)	Estado
configuration	JSON	Configuración adicional
created_at	TIMESTAMP	Creación
updated_at	TIMESTAMP	Actualización

deleted_at

TIMESTAMP

Borrado lógico

6. Diferencia entre orden y jerarquía

No se debe utilizar una sola columna para representar ambas cosas.

6.1. Orden

Indica en qué posición se muestra un atributo.

Ejemplo:

1. Anime
2. Aldea
3. Tipo de chakra
4. Rango
5. Habilidades

Para eso sirve:

sort_order

6.2. Jerarquía

Indica que un atributo depende conceptualmente de otro.

Ejemplo:

Obra

- └─ Anime
 - └─ Tipo de chakra
 - └─ Aldea ninja
 - └─ Rango ninja

Para eso no basta con:

hierarchy_level = 2

El nivel solamente indica profundidad, pero no identifica quién es el padre.

Se necesita una relación real.

7. Tabla **attribute_relationships**

7.1. Propósito

Representa relaciones entre atributos.

Un atributo puede estar relacionado con muchos otros atributos.

Esta estructura permite relaciones flexibles, no solamente una jerarquía rígida.

7.2. Campos

Campo	Tipo	Descripción
id	BIGINT, PK	Identificador

parent_attribute_id	BIGINT, FK	Atributo origen o superior
child_attribute_id	BIGINT, FK	Atributo dependiente
relationship_type	VARCHAR(30)	Tipo de relación
is_required	BOOLEAN	Relación obligatoria
sort_order	INT	Orden dentro del padre
depth_level	INT	Nivel orientativo
configuration	JSON	Configuración adicional
created_at	TIMESTAMP	Creación
updated_at	TIMESTAMP	Actualización

7.3. Restricción

```

UNIQUE(
  parent_attribute_id,
  child_attribute_id,
  relationship_type
)

```

8. Tipos de relaciones entre atributos

8.1. PARENT_CHILD

Representa una jerarquía general.

Ejemplo:

Identidad ficticia

```

└─ Anime
   └─ Tipo de chakra

```

8.2. DEPENDS_ON

El atributo hijo necesita que exista el atributo padre.

Ejemplo:

Tipo de chakra

DEPENDS_ON

Anime

La entidad no podría registrar **Tipo de chakra** si no tiene primero el atributo **Anime**.

8.3. REQUIRES

Un atributo obliga a utilizar otro.

Ejemplo:
Transformación avanzada
REQUIRES
Elemento base

8.4. EXCLUDES

Dos atributos no pueden utilizarse juntos.

Ejemplo:
Ser vivo
EXCLUDES
Objeto inanimado

Debe utilizarse con cuidado, porque OmniMerge busca ser creativo y no imponer demasiadas restricciones.

8.5. RELATED_TO

Solo representa una asociación conceptual.

Ejemplo:
Poder
RELATED_TO
Resistencia

No impone ninguna condición.

8.6. INHERITS_FROM

Un atributo puede heredar configuración de otro.

Ejemplo:
Poder mágico
INHERITS_FROM
Poder

Podría heredar:

- Tipo numérico.
 - Rango de 0 a 100.
 - Unidad.
 - Configuración visual.
-

8.7. GROUPS_WITH

Indica afinidad para presentación.

Ejemplo:
Fuerza
GROUPS_WITH
Resistencia

Aunque para la vista sigue siendo preferible utilizar grupos de atributos.

9. El ejemplo de Anime y Tipo de chakra

El caso puede representarse así:

Atributo principal

Atributo: Anime

Tipo: OPTION

Origen: CATALOG

Permite múltiples: Sí

Opciones del atributo Anime

Naruto

Dragon Ball

One Piece

Bleach

Atributo condicionado

Atributo: Tipo de chakra

Tipo: OPTION

Origen: CATALOG

Permite múltiples: Sí

Opciones

Fuego

Agua

Tierra

Viento

Rayo

Relación

Anime

↓

Tipo de chakra

Sin embargo, aquí falta una condición importante:

Tipo de chakra solamente debe aparecer cuando el valor de Anime sea Naruto.

Eso no es únicamente una relación entre atributos. Es una relación condicionada por una opción.

10. Tabla **attribute_conditions**

10.1. Propósito

Define cuándo un atributo debe mostrarse, habilitarse, ocultarse u obligarse según el valor de otro atributo.

10.2. Campos

Campo	Tipo	Descripción
id	BIGINT, PK	Identificador
target_attribute_id	BIGINT, FK	Atributo afectado
source_attribute_id	BIGINT, FK	Atributo evaluado
source_option_id	BIGINT, FK, NULL	Opción evaluada
operator	VARCHAR(30)	Operador
action	VARCHAR(30)	Acción
comparison_value	JSON	Valor a comparar
priority	INT	Prioridad
status	VARCHAR(20)	Estado
created_at	TIMESTAMP	Creación
updated_at	TIMESTAMP	Actualización

10.3. Operadores

EQUALS

NOT_EQUALS

CONTAINS

NOT_CONTAINS

GREATER_THAN

LESS_THAN

HAS_VALUE

HAS_NO_VALUE

IN

NOT_IN

10.4. Acciones

SHOW

HIDE

ENABLE

DISABLE

REQUIRE

MAKE_OPTIONAL

FILTER_OPTIONS

10.5. Ejemplo

Atributo evaluado:

Anime

Opción evaluada:

Naruto

Operador:

EQUALS

Atributo afectado:

Tipo de chakra

Acción:

SHOW

La regla completa sería:

SI Anime contiene Naruto

ENTONCES mostrar Tipo de chakra.

11. No es recomendable depender del nombre

La condición no debe almacenarse así:

SI Anime = "Naruto"

porque el usuario podría cambiar el nombre a:

Naruto Shippuden

Debe utilizar identificadores:

source_attribute_id = 10

source_option_id = 85

target_attribute_id = 24

Así la relación continúa funcionando aunque el texto visible cambie.

12. Dependencia según varias opciones

Un atributo puede habilitarse con diferentes valores.

Ejemplo:

Sistema de energía

puede mostrarse cuando la obra sea:

Naruto

Dragon Ball
Bleach

Pero sus opciones podrían variar:

Naruto:

- Chakra

Dragon Ball:

- Ki

Bleach:

- Energía espiritual

Para esto se pueden crear varias condiciones:

Anime = Naruto

→ mostrar Sistema de energía

Anime = Dragon Ball

→ mostrar Sistema de energía

Anime = Bleach

→ mostrar Sistema de energía

13. Filtrado dinámico de opciones

Puede ocurrir que el mismo atributo tenga diferentes opciones según otro valor.

Ejemplo:

Atributo: Técnica

Si:

Anime = Naruto

mostrar:

Rasengan

Chidori

Byakugan

Si:

Anime = Dragon Ball

mostrar:

Kamehameha

Genki Dama

Kaioken

Para esto se necesita relacionar opciones con condiciones.

14. Tabla **attribute_option_rules**

14.1. Propósito

Permite definir qué opciones estarán disponibles según otro atributo u opción.

14.2. Campos

Campo	Tipo	Descripción
id	BIGINT, PK	Identificador
target_option_id	BIGINT, FK	Opción que será permitida
source_attribute_id	BIGINT, FK	Atributo evaluado
source_option_id	BIGINT, FK	Opción necesaria
operator	VARCHAR(30)	Operador
action	VARCHAR(30)	Acción
priority	INT	Prioridad
created_at	TIMESTAMP	Creación
updated_at	TIMESTAMP	Actualización

14.3. Ejemplo

SI Anime = Naruto

PERMITIR Técnica = Rasengan

SI Anime = Naruto

PERMITIR Técnica = Chidori

SI Anime = Dragon Ball

PERMITIR Técnica = Kamehameha

Esto permite que un atributo posea un catálogo general, pero muestre únicamente las opciones adecuadas al contexto.

15. Relación de atributos con tipos de entidad

También es necesario definir en qué tipos de entidad se recomienda utilizar un atributo.

Tabla:

entity_type_attributes

Campos:

Campo	Tipo	Descripción
entity_type_id	BIGINT, FK	Tipo de entidad
attribute_id	BIGINT, FK	Atributo
is_required	BOOLEAN	Obligatorio
is_suggested	BOOLEAN	Sugerido
default_group_id	BIGINT, FK	Grupo visual
sort_order	INT	Orden
configuration	JSON	Configuración

Ejemplo:

Tipo de entidad: Personaje de anime

Atributos sugeridos:

- Anime
- Raza
- Habilidades
- Sistema de energía

Pero el usuario podría agregar otros atributos.

16. Grupos y jerarquías no son lo mismo

Grupo

Se utiliza principalmente para organizar la vista.

Ejemplo:

Información de la obra

- Anime
- Temporada
- Autor

Información de combate

- Tipo de chakra
- Técnicas
- Poder

Jerarquía

Representa dependencia conceptual.

Ejemplo:

Anime

└─ Tipo de chakra

Condición

Define cuándo se utiliza.

Ejemplo:

Si Anime = Naruto

mostrar Tipo de chakra.

Por tanto, deben existir estructuras diferentes:

attribute_groups

attribute_group_attribute

attribute_relationships

attribute_conditions

17. Jerarquía múltiple

Mencionas que un atributo puede tener varias jerarquías o relaciones.

Por ejemplo:

Tipo de chakra

puede depender de:

Anime

Sistema de energía

Tipo de personaje

Por eso no conviene colocar simplemente:

parent_attribute_id

dentro de la tabla **attributes**.

Esa columna permitiría un único padre.

Es mejor utilizar:

attribute_relationships

Así se puede representar:

Anime → Tipo de chakra

Sistema de energía → Tipo de chakra

Personaje ninja → Tipo de chakra

El mismo atributo puede tener varios padres o dependencias.

18. Combinación de condiciones

Las condiciones pueden combinarse.

Ejemplo:

Mostrar Kekkei Genkai si:

Anime = Naruto

Y

Tipo de personaje = Ninja

Se necesitaría agregar:

condition_group

a **attribute_conditions**.

Ejemplo:

Grupo 1:

Anime = Naruto

AND

Tipo de personaje = Ninja

También se podría permitir:

Anime = Naruto

OR

Anime = Boruto

Para ello pueden agregarse:

Campo	Descripción
condition_group	Agrupación de condiciones
logical_operator	AND u OR
priority	Orden de evaluación

19. Tabla ampliada **attribute_conditions**

Una versión más completa sería:

Campo	Tipo
id	BIGINT
target_attribute_id	BIGINT
source_attribute_id	BIGINT
source_option_id	BIGINT, NULL
operator	VARCHAR(30)
action	VARCHAR(30)

comparison_value	JSON
condition_group	INT
logical_operator	VARCHAR(10)
priority	INT
error_message	VARCHAR(255)
status	VARCHAR(20)
created_at	TIMESTAMP
updated_at	TIMESTAMP

20. Atributos globales y privados

Conviene definir el alcance del atributo.

Agregar a **attributes**:

scope

Valores:

PRIVATE

SHARED

PUBLIC

SYSTEM

PRIVATE

Solo puede utilizarlo su creador.

SHARED

Puede compartirlo con usuarios específicos.

PUBLIC

Otros usuarios pueden reutilizarlo.

SYSTEM

Es un atributo creado por OmniMerge.

Para la primera versión pueden utilizarse solamente:

PRIVATE

SYSTEM

21. Duplicar o reutilizar atributos

Cuando un usuario quiera usar un atributo público de otra persona, existen dos posibilidades:

Reutilización directa

Ambos utilizan el mismo atributo.

Problema:

El creador podría modificarlo y afectar a otros usuarios.

Crear una copia

El usuario importa el atributo a su biblioteca.

Ventaja:

Puede modificarlo sin afectar al original.

Se recomienda permitir:

Clonar atributo

Agregar en **attributes**:

source_attribute_id

Esto permitirá conocer de qué atributo fue copiado.

22. Versionado de atributos

Si un atributo ya se utiliza en muchas entidades, cambiar su tipo puede producir errores.

Ejemplo:

Poder:

Antes: INTEGER

Después: TEXT

Los valores anteriores dejarían de ser compatibles.

Debe establecerse la regla:

Un atributo utilizado no puede cambiar libremente su tipo de dato.

Opciones:

1. Bloquear el cambio.
2. Crear una nueva versión.
3. Solicitar conversión de valores.
4. Duplicar el atributo.

Para una primera versión se recomienda bloquear cambios de **data_type** cuando ya existan valores.

23. Restricciones entre opciones

También puede existir incompatibilidad entre opciones.

Ejemplo:

Estado vital:

- Vivo
- Muerto

No deberían seleccionarse ambos al mismo tiempo.

Otro ejemplo:

Elemento:

- Fuego
- Agua

Podrían ser incompatibles en un universo específico, pero compatibles en otro.
Para esto se puede crear una tabla futura:
attribute_option_relationships

Campos:

Campo	Descripción
source_option_id	Opción origen
target_option_id	Opción relacionada
relationship_type	Tipo
configuration	Reglas

Tipos:

COMPATIBLE_WITH
INCOMPATIBLE_WITH
REQUIRES
UPGRADES_TO
DERIVED_FROM
RELATED_TO

24. Opciones dependientes de otras opciones

Ejemplo:

País:

- Perú
- Japón

Ciudad:

- Tacna
- Lima
- Tokio
- Osaka

Reglas:

Perú

└─ Tacna
└─ Lima

Japón

└─ Tokio
└─ Osaka

Este caso puede modelarse de dos formas:

Forma sencilla

Con `parent_option_id`, cuando ambas opciones pertenecen a una misma estructura.

Forma flexible

Con `attribute_option_rules`, cuando las opciones pertenecen a atributos diferentes.

Ejemplo:

País = Perú

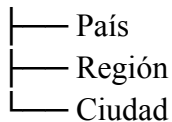
→ permitir Ciudad = Tacna

25. Atributos compuestos

Algunos atributos podrían contener subatributos.

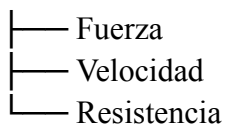
Ejemplo:

Ubicación



Otro:

Estadísticas físicas



En muchos casos esto debe manejarse mediante grupos, no jerarquías.

Regla recomendada:

- Si es solo para organizar la vista: utilizar grupos.
- Si un atributo depende realmente de otro: utilizar relaciones.
- Si se muestra según un valor: utilizar condiciones.
- Si es una opción inferior de otra opción: utilizar jerarquía de opciones.

26. Atributos repetibles

Puede haber atributos estructurados que se repitan.

Ejemplo:

Transformaciones:

Transformación 1:

- Nombre: Modo Sabio
- Poder: 85
- Duración: 15 minutos

Transformación 2:

- Nombre: Modo Kurama
- Poder: 100

- Duración: 10 minutos

Esto no se representa bien con valores simples.

En el futuro podría añadirse:

REPEATER

como tipo de atributo.

Su configuración se guardaría en JSON o mediante una estructura de bloques.

No es recomendable implementarlo en el primer MVP, pero sí dejarlo previsto.

27. Campos adicionales recomendados

En **attributes**

Agregar:

help_text

placeholder

display_style

validation_rules

scope

source_attribute_id

help_text

Texto de ayuda.

Seleccione una o varias naturalezas de chakra.

placeholder

Ejemplo visual.

Ejemplo: Fuego, Agua o Viento

display_style

Cómo se muestra el campo:

SELECT

MULTISELECT

RADIO

CHECKBOX

TAGS

SLIDER

TEXTBOX

TEXTAREA

COLOR_PICKER

DATE_PICKER

validation_rules

JSON con reglas adicionales.

Ejemplo:

```
{  
  "minimumSelections": 1,  
  "maximumSelections": 3  
}
```

28. Estructura final recomendada

Tablas principales

users

entity_types

entities

attributes

attribute_options

entity_attributes

entity_attribute_values

Organización visual

attribute_groups

attribute_group_attribute

collections

collection_entity

Relaciones y dependencias

attribute_relationships

attribute_conditions

attribute_option_rules

Recomendaciones por tipo

entity_type_attributes

Futuras relaciones avanzadas

attribute_option_relationships

29. Relaciones generales

User

├── EntityTypes

├── Entities

├── Attributes

└── AttributeGroups

└─ Collections

Attribute

└─ AttributeOptions
└─ EntityAttributes
└─ AttributeRelationships
└─ AttributeConditions
└─ AttributeGroups
└─ EntityTypes

Entity

└─ EntityAttributes
 └─ EntityAttributeValue

30. Ejemplo integral

Entidad

Nombre: Naruto Uzumaki

Tipo: Personaje de anime

Atributos generales

Obra:

- Naruto

Tipo de personaje:

- Ninja

Aldea:

- Aldea Oculta de la Hoja

Condiciones

SI Obra = Naruto

MOSTRAR Tipo de chakra

SI Tipo de personaje = Ninja

MOSTRAR Rango ninja

SI Obra = Naruto

Y Tipo de personaje = Ninja

MOSTRAR Kekkei Genkai

Valores resultantes

Tipo de chakra:

- Viento

Rango ninja:

- Genin

Kekkei Genkai:

- Ninguno

Grupos visuales

[Información de la obra]

- Obra
- Tipo de personaje

[Información ninja]

- Aldea
- Rango ninja
- Tipo de chakra
- Kekkei Genkai

[Combate]

- Poder
- Velocidad
- Técnicas

31. Qué debe implementarse primero

No se recomienda implementar inmediatamente todas las reglas avanzadas.

MVP 1: núcleo

attributes

attribute_options

entity_attributes

entity_attribute_values

Debe permitir:

- Atributos libres.
- Catálogos seleccionables.
- Valores únicos.
- Valores múltiples.
- Orden.
- Grupos visuales.

MVP 2: dependencias

attribute_relationships

attribute_conditions

Debe permitir:

- Atributo padre e hijo.
- Mostrar atributos según opciones.
- Requerir atributos.
- Ocultar atributos no aplicables.

MVP 3: reglas avanzadas

attribute_option_rules

attribute_option_relationships

Debe permitir:

- Filtrar opciones por contexto.
- Incompatibilidades.
- Requisitos entre opciones.
- Catálogos dependientes.

32. Conclusión

La tabla de valores seleccionables es necesaria, pero conviene llamarla **attribute_options** y no **adjective**, porque almacenará cualquier tipo de opción, no solamente calificativos.

La jerarquía tampoco debe representarse únicamente mediante una columna numérica.

La columna:

sort_order

debe utilizarse para el orden visual.

La columna:

hierarchy_level

puede utilizarse como información auxiliar, pero no debe ser la relación principal.

Las relaciones reales deben almacenarse mediante:

attribute_relationships

Las reglas basadas en valores deben almacenarse mediante:

attribute_conditions

Y el filtrado de opciones debe almacenarse mediante:

attribute_option_rules

De esta manera, OmniMerge podrá representar estructuras como:

Anime

```

├── Naruto
│   ├── Tipo de chakra
│   ├── Aldea ninja
│   ├── Rango ninja
│   └── Técnicas

```

sin convertirlas en columnas rígidas de la base de datos.

El sistema seguirá siendo completamente creativo, porque será el usuario quien defina:

- Los atributos.
- Sus opciones.
- Sus grupos.
- Sus relaciones.
- Sus dependencias.
- Sus condiciones.
- El orden de visualización.
- Los atributos aplicables a cada entidad.